

Exercices de spécification, programmation et preuve de fonctions WhyML

Enseignement à distance de l'Université Marie et Louis Pasteur
Spécification et preuve de programmes

Version du 9 décembre 2025

1 Préconditions et postconditions

Annoter les fonctions du listing 1 avec des pré- et postconditions précises et pertinentes, en tenant compte des indications suivantes :

1. Pour un débit, on ne fait que *retirer* de l'argent,
2. à l'inverse, pour un crédit, on ne fait qu'en *ajouter*,
3. les diverses opérations doivent être correctes,
4. le montant du compte doit être positif à tout instant,
5. un compte contient une quantité d'argent maximale, il faut s'assurer que cette quantité ne soit pas dépassée.

```
use int.Int
use ref.Ref

let constant max_balance : int = 1000

val balance : ref int

let debit (amount : int) : int
=
  balance := !balance - amount;
  !balance

let credit (amount : int) : int
=
  balance := !balance + amount;
  !balance

let getBalance () : int
=
  !balance
```

Listing 1 – Fichier Why3 pour un compte bancaire.

2 Premiers contrats

Dans cet exercice, il s'agit d'ajouter des annotations à du code WhyML fourni et réciproquement. Toutes les réponses doivent être dans le même fichier. Utiliser l'interface en ligne de Why3 pour vérifier la syntaxe et prouver la conformité du code avec les annotations. Corriger le code ou les annotations jusqu'à ce qu'il n'y ait plus aucune erreur.

2.1 Valeur absolue d'un entier

1. Écrire une fonction conforme au contrat ci-dessous.

```
let absVal (x:int) :int
  ensures { x ≥ 0 → result = x }
  ensures { x < 0 → result = -x }
=
  ...
```

2. Prouver la conformité contrat-programme avec Why3.

2.2 Incrémentation d'un entier

1. Écrire le contrat le plus précis possible pour la fonction

```
let incr (n:ref int) : int
=
  n := !n+1;
  !n
```

2. Prouver la conformité entre ce programme et les postconditions ajoutées.

2.3 Maximum de deux entiers

1. Vérifier la conformité entre le contrat et le programme de cette fonction :

```
let maxint (x y:int) : int
  ensures { x > y → result = x }
  ensures { x ≤ y → result = 0 }
=
  if x > y then x else y
```

La preuve devrait échouer car la conformité n'est pas satisfaite.

2. Corriger le contrat ou le programme d'après la définition que vous donnez "naturellement" au calcul du maximum de deux entiers. Vérifier ensuite que la preuve réussit.

2.4 Maximum de trois entiers x , y et z différents

1. Vérifier la conformité contrat-programme pour la fonction `max3int` définie comme suit :

```
let max3int(x y z:int) : int
  requires { x ≠ y ∧ x ≠ z ∧ y ≠ z }
  ensures { x > y ∧ x > z → result = x }
  ensures { y > x ∧ y > z → result = y }
  ensures { z > x ∧ z > y → result = z }
=
  if x > y then
    if x > z then x else z
  else
    if y > z then y else x
```

2. La preuve devrait échouer car la conformité n'est pas satisfaite. Corriger le contrat ou le programme d'après la définition que vous donnez "naturellement" à cette fonction. Vérifier ensuite que la preuve réussit.

2.5 Échange de deux valeurs arithmétiques

1. Exprimer par une postcondition WhyML que la fonction `arithSwap` suivante échange les valeurs référencées par les références `x` et `y`. Prouvez ensuite votre postcondition avec Why3.

```
let arithSwap (ref x y: int) : unit
=
  x := x + y;
  y := x - y;
  x := x - y
```

2. Mêmes questions avec une implémentation classique de l'échange, sans arithmétique, mais à l'aide d'une troisième variable.

3 Racine carrée entière par défaut par décrémentation

1. Définir en WhyML une fonction `sqrtDec` qui calcule la racine carrée entière par défaut de son paramètre n , par décrémentation, selon l'algorithme suivant :

$$r := n; \text{ while } r^2 > n \text{ do } r := r - 1 \text{ od}$$

2. Spécifier le contrat de cette méthode par une pré- et une postcondition en WhyML.
3. Spécifier un invariant de boucle assez précis pour permettre de démontrer cette postcondition.
4. Faire faire la preuve de conformité contrat-programme par Why3, uniquement la preuve de correction partielle en l'indiquant par l'annotation `diverges`.

4 Multiplication par additions

La fonction du listing 2 calcule le produit de ses deux paramètres x et y dans la variable globale référencée par p .

```
use int.Int
use ref.Ref

val ref p : int

let mult (x y:int) : unit
  requires { ... }
  ensures { ... }
  diverges
=
  let ref i = 0 in
  p := 0;
  while i < x do
    invariant { ... }
    i := i + 1;
    p := p + y
  done
```

Listing 2 – Multiplication par additions en WhyML.

1. Ajouter une précondition WhyML définissant le plus grand ensemble de données pour lequel le programme est applicable pour obtenir un produit correct.
2. Ajouter une postcondition WhyML définissant la valeur de la variable référencée par p .
3. Ajouter un invariant de boucle pour prouver la postcondition avec Why3.
4. Quand la preuve de correction partielle a réussi, supprimer la clause `diverges` et refaire la preuve. Que constatez-vous ? Ajouter une clause `variant` à la boucle, pour prouver la terminaison. Le variant doit être une expression entière positive ou nulle qui décroît strictement à chaque pas d'itération.
5. Généraliser le programme afin d'enlever totalement la précondition et de traiter tous les entiers relatifs x et y . Faire sa preuve.